MycoPortal Agent

Natural Language Fungal Biodiversity Agent • MycoPortal / Symbiota API

Table of Contents

1. Overview 4
   1.1 Supported Intents and Endpoints 4
2. Technology Stack 4
   2.1 Runtime and Language 4
   2.2 AI / LLM Layer 4
   2.3 Data Validation 5
   2.4 HTTP Client 5
   2.5 Testing Infrastructure 5
3. Architecture 5
   3.1 Five-Stage Pipeline 5
   3.2 Stage Detail - Planner 6
   3.3 Stage Detail - Extractor 6
   3.4 Stage Detail - Router 6
   3.5 Stage Detail - Executor 7
   3.6 Project Layout 7
4. Configuration 8
   4.1 Config Resolution Order 8
   4.2 Required Keys 8
   4.3 env.yaml Example 8
5. Installation and Running 8
   5.1 Prerequisites 8
   5.2 Install 8
   5.3 Run 9
   5.4 Example Queries 9
6. Data Validation Layer 10
   6.1 Extractor-Level Validation (LLM-guided) 10
   6.2 Router / Param-Model Validation (Pydantic) 10
7. Testing 11
   7.1 Framework and Markers 11
   7.2 Test Dataset System 11
   7.3 Test Buckets 11
   7.4 Running Tests 12
8. Test Cases - Complete Reference 12
   8.1 Occurrence Search 12
      Happy-Path Cases 12
      Clarification Cases 14
      Edge Case 14

8.2 Occurrence By ID 14
Happy-Path Cases 14
Clarification Cases 15
Validation Failure Case 15
Expected Failure Case 15
8.3 Taxonomy Search 16
Search Type Routing Cases 16
EXACT / Default Cases 16
Expected Failure Case 17
8.4 Taxon By ID 17
Happy-Path Cases 17
Clarification Cases 18
8.5 Collection List 18
Happy-Path Cases 18
Clarification Case 19
8.6 Media Lookup 19
Happy-Path Cases 19
Clarification Case 20
8.7 Morphology List 20
8.8 Exsiccata List 21
9. DeepEval Quality Evaluation 21
9.1 Metrics 21
9.2 Score Distribution 21
9.3 Threshold Override 21
10. Artifact System 22
10.1 Artifact Metadata 22
10.2 Functional Artifact Assertions 22
11. Test Fixture - FakeResponseContext 22
12. Extending the Agent - Adding a New Endpoint 23

1. Overview

MycoPortal Agent is a natural-language interface to the MycoPortal (Symbiota network) fungal biodiversity data platform. It is built on the iChatBio agent framework and exposes a five-stage LLM-driven pipeline that converts a free-text user query into structured API calls, then packages the results as downloadable data artifacts.

What the agent does
A user types a plain-English question - for example, "Find Ganoderma occurrences in Florida" - and the agent automatically selects the right MycoPortal endpoint, constructs the correct query parameters, fetches the data, and returns a JSON artifact alongside a human-readable reply. No API knowledge is required from the user.

1.1 Supported Intents and Endpoints

Intent	HTTP Endpoint	Description
occurrence_search	GET /api/v2/occurrence/search	Paginated specimen-level occurrence search
occurrence_by_id	GET /api/v2/occurrence/{id}	Single occurrence lookup by UUID
taxonomy_search	GET /api/v2/taxonomy/search	Taxon name search with match-type control
taxon_by_id	GET /api/v2/taxonomy/{id}	Full taxon record by numeric tid
collection_list	GET /api/v2/collection	Paginated institutional collection listing
media_lookup	GET /api/v2/media	Media records linked to taxa / occurrences
morphology_list	GET /api/v2/morphology	Morphological character definitions
exsiccata_list	GET /api/v2/exsiccata	Published dried-specimen series listing

2. Technology Stack

2.1 Runtime and Language

Component	Version / Value	Notes
Python	3.10+	Minimum supported version
Async runtime	asyncio	All pipeline stages are async
HTTP server	uvicorn	ASGI server; port 9998
Agent SDK	ichatbio-sdk 0.2.1	iChatBio platform integration

2.2 AI / LLM Layer

Component	Package	Role
OpenAI-compatible LLM	openai 1.93.0	Inference for Planner and Extractor
Structured output	instructor 1.9.0	Pydantic-validated LLM responses via function calling
Default model	gpt-4o-mini	Used in both Planner and Extractor stages

2.3 Data Validation

Component	Package	Role
Schema models	pydantic 2.11.7	Extraction schemas and API param models
Field validators	pydantic v2	UUID format, binomen format, catalog number, date formats
Enum constraints	pydantic Literal	Search type, management type, collection type

2.4 HTTP Client

The MycoPortalAPI class in client/api.py uses the synchronous requests library (Requests 2.32.4) executed in a thread pool via asyncio.get_event_loop().run_in_executor(). This preserves the async pipeline while allowing simple synchronous HTTP calls with session-level headers and configurable timeouts.

2.5 Testing Infrastructure

Component	Package	Role
Test runner	pytest 8.3.5	Parametrized async test execution
Async support	pytest-asyncio 0.24.0	asyncio_mode = auto
Semantic quality	deepeval >=3.9.5,<4.0.0	GEval metrics for reasoning and extraction quality
Reporting	allure-pytest 2.13.5	Allure HTML report generation
Data viz	matplotlib >=3.9, numpy	Score distribution histogram generation

3. Architecture

3.1 Five-Stage Pipeline

Every user request flows through five ordered stages. Stages 1–4 are LLM-assisted; Stage 5 is deterministic.

Pipeline Flow

User Query → [1] Planner → [2] Extractor → [3] Resolver → [4] Router → [5] Executor → Artifact + Reply

Stage	Module	Input	Output	LLM?
1. Planner	planning/planner.py	Raw query string	PlannerOutput (intent, tools, reasoning)	Yes
2. Extractor	extraction/extractor.py	Query + PlannerOutput	Typed ExtractionResult	Yes
3. Resolver	resolution/resolver.py	ExtractionResult	ResolutionResult (v1 passthrough)	No
4. Router	routing/router.py	Plan + Extraction + Res.	dict[tool_name → API param model]	No
5. Executor	execution/executor.py	Plan + routed params	JSON artifact + reply via context	No

3.2 Stage Detail - Planner

The Planner receives the raw query and selects exactly one intent from the supported set. It uses a system prompt that embeds all tool capability documents from planning/capabilities/*.md via the ToolCapabilityRegistry. The output is a PlannerOutput Pydantic model, enforced by instructor, containing the intent, a list of ToolPlan entries (each tagged must_call or optional), a reasoning string, and optional clarification fields.

Key constraints enforced by Pydantic model validators: at least one must_call tool is required for any non-scope intent; if clarification_needed is true, a clarification_question must also be present.

3.3 Stage Detail - Extractor

The Extractor maps the planner's intent to a schema class (INTENT_SCHEMA_MAP), then calls the LLM again to fill that schema from the query text. The system prompt includes detailed extraction rules covering ID formats, scientific name conventions, catalog number patterns, date formats, county constraints, and search type selection.

All schema classes inherit from ExtractionBase, which provides the clarification_needed / clarification_question escape hatch. If the extractor cannot confidently populate required fields (e.g., a UUID is malformed or a taxon ID is non-numeric), it sets clarification_needed = True instead of guessing.

3.4 Stage Detail - Router

The Router is fully deterministic. It receives the plan intent and the typed extraction object and constructs the corresponding API parameter model. Field name translation from extraction snake_case to API camelCase happens here (e.g., state_province → stateProvince). Pydantic field validators on the param models provide a second validation pass - for instance, sciname is re-validated against the binomen regex pattern at routing time.

3.5 Stage Detail - Executor

The Executor iterates tools in priority order: all must_call tools are executed first; optional tools follow. For each tool it dispatches to the appropriate tool module in execution/tools/, which builds the URL, fires the HTTP GET, and creates a JSON artifact via the shared create_json_artifact helper. Failures in must_call tools abort the pipeline; failures in optional tools are logged but do not stop execution.

3.6 Project Layout

```
MycoPortal_iChatBio/
├── agent.py          # Agent orchestration and pipeline wiring
├── server.py         # uvicorn HTTP server (port 9998)
├── requirements.txt
├── env.yaml          # Runtime config (gitignored)
├── common/
│   ├── config.py     # Env-var + YAML config loader
│   └── types.py      # Re-exported Pydantic types
├── client/
│   └── api.py        # MycoPortal HTTP client + URL builders
├── planning/
│   ├── planner.py    # LLM planner (gpt-4o-mini + instructor)
│   ├── models.py     # PlannerOutput, ToolPlan Pydantic models
│   ├── registry.py   # Loads capabilities/*.md for planner prompt
│   └── capabilities/ # One .md per tool - tool capability docs
├── extraction/
│   ├── extractor.py  # LLM extractor (gpt-4o-mini + instructor)
│   └── models.py     # Extraction schema classes
├── resolution/
│   └── resolver.py   # Passthrough resolver (v1 placeholder)
├── routing/
│   └── router.py     # Deterministic extraction → param mapping
├── execution/
│   ├── executor.py   # Tool priority orchestration
│   └── tools/        # One module per tool (artifact creation)
├── models/
│   └── params.py     # Pydantic API param models with validators
└── tests/
    ├── conftest.py   # Shared fixtures (pipeline, dataset, client)
    ├── data/         # JSON test datasets per endpoint
    ├── fixtures/     # FakeResponseContext mock
    └── integration/  # Functional + quality test modules
```

4. Configuration

4.1 Config Resolution Order

The get_config_value() function in common/config.py first checks OS environment variables, then falls back to env.yaml in the repository root. env.yaml is gitignored and must be created locally.

4.2 Required Keys

Key	Required	Default	Description
OPENAI_API_KEY	Yes	-	API key for the LLM provider
OPENAI_BASE_URL	No	https://api.openai.com/v1	Override for custom or proxy endpoints
MYCOPORTAL_API_BASE_URL	No	https://mycoportal.org/portal	MycoPortal base URL
MYCOPORTAL_HTTP_TIMEOUT	No	30	HTTP timeout in seconds

4.3 env.yaml Example

```
OPENAI_API_KEY: sk-...
OPENAI_BASE_URL: https://api.openai.com/v1
MYCOPORTAL_API_BASE_URL: https://mycoportal.org/portal
MYCOPORTAL_HTTP_TIMEOUT: 30
```

5. Installation and Running

5.1 Prerequisites

- Python 3.10 or later
- pip (or a virtual-environment manager such as venv or conda)
- A valid OPENAI_API_KEY (or compatible endpoint)

5.2 Install

```
# Clone the repository
git clone <repo-url>
cd MycoPortal_iChatBio

# Create and activate a virtual environment (recommended)
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
```

```
# Install dependencies
pip install -r requirements.txt

# Create env.yaml with your credentials
cp env.yaml.example env.yaml   # then edit with your keys
```

5.3 Run

```
python server.py
# Server starts at http://0.0.0.0:9998
```

5.4 Example Queries

Query	Intent triggered
Find Ganoderma occurrences in Florida	occurrence_search
Show occurrence records for Amanita muscaria	occurrence_search
Search taxonomy for names starting with Aspergillus	taxonomy_search
What is amanita	taxonomy_search
Get taxon 388665	taxon_by_id
Show me 50 live preserved-specimen collections from 100	collection_list
Lookup media for taxon id 388665	media_lookup
What morphological traits does MyCoPortal track?	morphology_list
List all exsiccata	exsiccata_list
Show occurrence b1e1f38f-2873-4d98-bf34-404dd97074d4	occurrence_by_id

6. Data Validation Layer

Validation happens at two independent points in the pipeline, providing defence-in-depth against malformed input.

6.1 Extractor-Level Validation (LLM-guided)

The Extractor system prompt contains explicit rules that instruct the LLM to set clarification_needed = True rather than guess when values are suspect. These rules are enforced by the instructor retry loop (max_retries = 2).

Field	Rule enforced by Extractor
occurrence_by_id identifier	Must match UUID format (8-4-4-4-12 hex with hyphens)
taxon_by_id identifier	Must be a positive integer tid
media_lookup tid	Must be numeric; alpha-numeric strings trigger clarification
catalog_number	Must match dashed portal format e.g. DBG-F-010127
county	Single county only; comma-separated values trigger clarification
sciname	Binomen only (Genus species); authorship suffix triggers clarification
eventDate	Must be YYYY, YYYY-MM, or YYYY-MM-DD
taxonomy search_type	START / WHOLEWORD / WILD selected by phrase analysis; not defaulted to EXACT

6.2 Router / Param-Model Validation (Pydantic)

The Pydantic models in models/params.py apply a second, deterministic validation pass using field_validator decorators. Even if the LLM extractor passes a value through, these validators will raise a ValidationError that the test suite can assert on.

Param model field	Validator / pattern
OccurrenceByIdParams.identifier	UUID_PATTERN regex (via Pydantic pattern=)
OccurrenceSearchParams.sciname	BINOMEN_PATTERN: ^[A-Z][A-Za-z-]* [a-z][a-z-]*$
OccurrenceSearchParams.catalogNumber	CATALOG_NUMBER_PATTERN: ^[A-Z0-9]+(?:-[A-Z0-9]+)+$
OccurrenceSearchParams.eventDate	Accepts YYYY | YYYY-MM | YYYY-MM-DD; parsed with datetime.strptime
OccurrenceSearchParams.county	Rejects values containing a comma
TaxonByIdParams.identifier	ge=1 (must be positive integer)
MediaLookupParams.tid	ge=1 (must be positive integer)
CollectionListParams.managementType	Literal['live','snapshot','aggregate']

7. Testing

7.1 Framework and Markers

Tests live in tests/integration/ and are executed with pytest. All tests are async (asyncio_mode = auto). Markers gate execution:

Marker	Meaning
functional	End-to-end pipeline checks, including live API calls
quality	DeepEval GEval semantic scoring for reasoning and extraction accuracy
smoke	Fast subset for PR validation
llm	Any test that calls an LLM endpoint
liveapi	Tests that call live MycoPortal endpoints
negative	Expected-failure intent-mismatch tests

7.2 Test Dataset System

Tests are data-driven. Test cases are stored as JSON files in tests/data/ and loaded at collection time. The loader supports three modes:

Mode	CLI flag	Dataset loaded
smoke	--dataset-mode smoke	eval_dataset.json (consolidated cross-endpoint set)
endpoint	--dataset-mode endpoint --endpoint <name>	Single endpoint file e.g. taxonomy_search_testdata.json
regression	--dataset-mode regression	All endpoint-specific JSON files combined

7.3 Test Buckets

Each test module contains four parametrized test functions, one per bucket:

Test function	Bucket	What it validates
test_live_pipeline_generates_artifact_for_case	functional_success	Full happy-path pipeline produces a valid artifact
test_expected_failure_case_mismatches_intent	expected_failure	Planner assigns a different intent than the (intentionally wrong) expected value
test_clarification_case_requests_followup	clarification	Planner or Extractor requests clarification for ambiguous/invalid input

Test function	Bucket	What it validates
test_validation_failure_detected	validation_failure	A specific validation failure is caught at the expected pipeline stage

7.4 Running Tests

```
# Smoke tests only (fastest - good for PRs)
pytest -m 'smoke and functional' --alluredir=allure-results

# Full functional + quality suite
pytest -m 'functional or quality' --alluredir=allure-results

# Single endpoint dataset
pytest --dataset-mode endpoint --endpoint taxonomy_search

# Full regression across all endpoints
pytest --dataset-mode regression

# Generate and open Allure HTML report
allure generate allure-results --clean -o allure-report
allure open allure-report
```

8. Test Cases - Complete Reference

The following sections list every unique test case across all endpoint datasets. Each entry explains what part of the design the test validates.

8.1 Occurrence Search

Happy-Path Cases

occurrence_search_sciname_only

Query: "Find occurrence records for Leccinum scabrum"

Design area validated: Extractor correctly strips any implied context and maps the species name to the sciname field. Pydantic BINOMEN_PATTERN validator confirms the value is Genus species format. Executor calls /api/v2/occurrence/search with sciname=Leccinum scabrum.

occurrence_search_state_only

Query: "Find fungal occurrences in Oregon"

Design area validated: Extractor maps a bare state name to stateProvince. Confirms that a single geographic filter is sufficient - the pipeline must not set clarification_needed even though no species is specified.

occurrence_search_county_only

Query: "Show species occurrences in Lane county"

Design area validated: County-only filter path. Validates that the Pydantic county validator accepts a single county name and that the executor correctly passes county= to the API.

occurrence_search_country_only

Query: "Find fungal occurrences in Canada"

Design area validated: Country-only filter. Confirms the router maps country correctly and the param model normalises whitespace via _normalize_optional_text.

occurrence_search_event_date_only - YYYY-MM

Query: "Show occurrences from 2024-05"

Design area validated: Partial date acceptance. The _validate_event_date validator must accept YYYY-MM and parse it with datetime.strptime("%Y-%m") without raising.

occurrence_search_full_date_iso - YYYY-MM-DD

Query: "Show occurrences from 2024-03-15"

Design area validated: Full ISO date path. Validator accepts YYYY-MM-DD and parses with "%Y-%m-%d".

occurrence_search_year_only_date

Query: "List all occurrences from 2023"

Design area validated: Year-only date. Validator accepts bare four-digit years matching \d{4}.

occurrence_search_recorded_by_only

Query: "Find occurrences recorded by Jane Doe"

Design area validated: recordedBy field path. Confirms full-name extraction and that the param model normalises whitespace.

occurrence_search_recorded_by_name

Query: "Find specimens collected by Smith"

Design area validated: Last-name-only collector path. Extractor may populate either recordedBy or recordedByLastName; router maps both correctly.

occurrence_search_catalog_number_valid

Query: "Find occurrences with catalog number DBG-F-010127"

Design area validated: Catalog number happy path. CATALOG_NUMBER_PATTERN regex ^[A-Z0-9]+(?:-[A-Z0-9]+)+$ must accept the dashed format.

occurrence_search_family_and_state

Query: "Find occurrences of family Boletaceae in Washington"

Design area validated: Multi-filter path. Extractor must populate both family and state_province; router maps both to the API model.

occurrence_search_max_offset_out_of_range

Query: "Find occurrences with offset 10000"

Design area validated: Large offset handling. OccurrenceSearchParams.offset has ge=0 but no upper bound, so a high value should pass validation and be forwarded to the API as-is.

Clarification Cases

occurrence_search_catalog_number_invalid_clarification

Query: "Find occurrences with catalog number DBG010127"

Design area validated: Extractor rule: catalog numbers missing hyphens should trigger clarification_needed = True. Tests the LLM's adherence to the catalog number format prompt instruction.

occurrence_search_sciname_with_authorship

Query: "Find occurrences for Amanita muscaria (L.)"

Design area validated: Authorship suffix rejection. Extractor must detect the parenthetical authorship string and request a binomen-only name instead of stripping silently.

occurrence_search_county_multi_value_invalid

Query: "Find occurrences with county 'Clear Creek, Douglas'"

Design area validated: Single-county constraint. Both the extractor prompt rule and the Pydantic _validate_county validator reject comma-containing values. This tests both layers.

Edge Case

occurrence_boletus_oregon_empty_results

Query: "Find occurrence records for Boletus edulis in Oregon"

Design area validated: Valid query that returns zero results from the live API. Tests that the executor and artifact creation handle an empty results array gracefully without raising.

8.2 Occurrence By ID

Happy-Path Cases

occurrence_by_id_valid_uuid

Query: "Show occurrence details for identifier 596a4eea-a8b9-40c5-a125-1288450fec7c"

Design area validated: Full UUID happy path. UUID_PATTERN regex and Pydantic pattern= field must accept a canonical 8-4-4-4-12 UUID. Executor calls /api/v2/occurrence/{id}.

occurrence_by_id_uppercase_hex

Query: "Show occurrence A1B2C3D4-E5F6-47A8-B9CD-EF1A2B3C4D5E"

Design area validated: Case-insensitive UUID. The UUID_PATTERN uses [0-9a-fA-F], confirming uppercase hex is accepted.

occurrence_by_id_uuid (eval_dataset)

Query: "Show occurrence details for identifier b1e1f38f-2873-4d98-bf34-404dd97074d4"

Design area validated: Live API integration. Full functional plus quality run; confirms the live endpoint returns a parseable JSON artifact.

Clarification Cases

occurrence_by_id_numeric

Query: "Get occurrence by identifier 1253291"

Design area validated: Integer-as-identifier rejection. The extractor must recognise that a bare integer is not a UUID and set clarification_needed = True.

occurrence_by_id_invalid_uuid_no_hyphens

Query: "Get occurrence by identifier 596a4eeaa8b940c5a1251288450fec7c"

Design area validated: UUID without hyphens. The extractor prompt rule states: if identifier is not in UUID format, set clarification_needed = True and ask for hyphen-separated format.

occurrence_by_id_invalid_random_identifier

Query: "Show occurrence details for identifier iufg637-r4d-123"

Design area validated: Malformed identifier with wrong segment structure. Extractor must detect non-UUID format (wrong segment count/lengths) and request a canonical UUID.

Validation Failure Case

occurrence_by_id_invalid_missing_segment

Query: "Get occurrence 596a4eea-a8b9-40c5-a125"

Design area validated: Param-model validation failure. Even if the extractor passes the truncated value, OccurrenceByIdParams.identifier's Pydantic pattern= constraint raises a ValidationError. Tests the test_validation_failure_detected bucket at validation_failure_point = param_schema_validation.

Expected Failure Case

occurrence_by_id_missing_identifier

Query: "Retrieve occurrence data using a generic identifier."

Design area validated: Planner handling of structurally valid but content-free queries. Marked should_fail=true; the test confirms the planner does not assign occurrence_by_id intent when no concrete identifier is present.

8.3 Taxonomy Search

Search Type Routing Cases

taxonomy_aspergillus_start

Query: "Search taxonomy for names starting with Aspergillus"

Design area validated: START search type selection. Phrase 'starting with' must trigger search_type = START per the extractor prompt mapping rules. Both intent and search type are asserted.

taxonomy_search_type_wholeword

Query: "Find taxonomy entries with the exact word boletus"

Design area validated: WHOLEWORD search type. Phrase 'exact word' maps to WHOLEWORD. Tests the extractor's phrase-to-type discrimination logic.

taxonomy_search_type_wild

Query: "Find taxonomy names containing muscaria"

Design area validated: WILD search type. Phrase 'containing' maps to WILD. Also validates that the router passes type=WILD to TaxonomySearchParams correctly.

EXACT / Default Cases

taxonomy_what_is_amanita

Query: "what is amanita"

Design area validated: Casual lookup phrasing resolves to taxonomy_search with EXACT type and taxon='amanita'. Tests that informal queries route correctly and do not trigger clarification.

taxonomy_coprinus_species

Query: "what species belong to coprinus"

Design area validated: Genus-membership phrasing. Tests that the planner selects taxonomy_search rather than occurrence_search for a taxonomic question about species membership.

taxonomy_ganoderma_genus

Query: "what kind of fungus is ganoderma"

Design area validated: Classification intent phrasing. Confirms the planner maps 'what kind of fungus is X' to taxonomy_search, not a factual lookup outside scope.

taxonomy_search_boletus_edulis_classification

Query: "what is boletus edulis classification"

Design area validated: Binomen + classification keyword. Tests that the extractor uses the full species name 'boletus edulis' as the taxon term and defaults to EXACT type.

taxonomy_search_tricholoma_mushrooms

Query: "taxonomy of tricholoma mushrooms"

Design area validated: Keyword 'taxonomy of' phrasing. The planner must select taxonomy_search intent and the extractor must extract 'tricholoma' not 'tricholoma mushrooms' as the taxon term.

validation_wrong_tool_in_plan

Query: "Search taxonomy for names starting with Aspergillus" (expected tool deliberately set to search_occurrences)

Design area validated: Validation-failure test for tool assertion. The planner will correctly select search_taxonomy, causing the must_call_tools_mismatch assertion to fire. Tests the test_validation_failure_detected bucket.

Expected Failure Case

negative_expected_failure_intent

Query: "Lookup media for taxon id 388665" (expected intent set to taxonomy_search)

Design area validated: Demonstrates the expected-failure bucket mechanics. The planner correctly selects media_lookup, which differs from the intentionally wrong expected_intent of taxonomy_search. Confirms the test framework correctly detects intent mismatches.

8.4 Taxon By ID

Happy-Path Cases

taxon_by_id_valid

Query: "Get taxon 388665"

Design area validated: Minimal numeric-ID path. Extractor extracts 388665 as integer; TaxonByIdParams.identifier ge=1 validates; executor calls /api/v2/taxonomy/388665.

taxon_by_id_with_details

Query: "show taxon details for id 95084"

Design area validated: Verbose phrasing with 'details for id' prefix. Tests that the extractor correctly parses the numeric tid from a more verbose prompt.

taxon_by_id_83373

Query: "what species is taxon 83373"

Design area validated: Question-form taxon lookup. Confirms the planner maps 'what species is taxon X' to taxon_by_id rather than taxonomy_search.

Clarification Cases

taxon_by_id_non_numeric

Query: "Get taxon with identifier abc123"

Design area validated: Alpha-numeric identifier rejection. The extractor prompt rule: taxon_by_id identifier must be numeric integer. An alphanumeric string triggers clarification.

taxon_by_id_float_invalid

Query: "Get taxon 123.45"

Design area validated: Float identifier rejection. Decimal values cannot be a valid integer tid. Extractor must detect the decimal and request a whole-number integer.

taxon_by_id_zero

Query: "Get taxon 0"

Design area validated: Zero-value rejection. TaxonByIdParams.identifier has ge=1, so zero is invalid. Tests whether the extractor catches this before the router or the Pydantic model raises.

taxon_by_id_very_large_number

Query: "Get taxon with identifier 9999999999999999999"

Design area validated: Overflow identifier. A 19-digit integer exceeds practical taxon ID ranges. Extractor should set clarification_needed or the Pydantic model may reject an integer that overflows Python int64 semantics.

8.5 Collection List

Happy-Path Cases

collection_list_live_specimens

Query: "Show me 50 live preserved-specimen collections starting from record 100"

Design area validated: Pagination plus type filter. Extractor must extract limit=50, offset=100, managementType=live, and collectionType=preservedSpecimens. All four fields must reach CollectionListParams.

collection_list_all_types

Query: "Show me all live preserved-specimen collections"

Design area validated: No explicit limit. Extractor uses default limit=1000. Tests managementType and collectionType extraction without pagination keywords.

collection_list_pagination_first_20

Query: "Show me the first 20 collections"

Design area validated: Limit-only pagination. No type filter present; extractor sets limit=20 and offset=0. Tests that missing optional filters do not cause clarification.

collection_list_range

Query: "List collections 50 through 100"

Design area validated: Range-to-offset conversion. '50 through 100' implies offset=50 and limit=50. Tests the extractor's arithmetic interpretation of range language.

collection_list_observation_only

Query: "List all observation collections"

Design area validated: Single collectionType filter. Extractor maps 'observation' to collectionType=observations. No managementType specified; router must omit the field (exclude_none=True).

Clarification Case

collection_observation_snapshot

Query: "List snapshot collections that contain observation data"

Design area validated: Ambiguous multi-filter intent. 'Snapshot' implies managementType and 'observation' implies collectionType, but the combination is semantically unclear. Marked clarification_stage=planner; the planner must ask for clarification rather than guess.

8.6 Media Lookup

Happy-Path Cases

media_by_taxon_id

Query: "Lookup media for taxon id 388665"

Design area validated: Basic tid-based media lookup. Extractor maps integer 388665 to tid field. Router sets includeSynonyms=0 and includeChildren=0 by default.

media_lookup_taxon_id_14501

Query: "Lookup media for tid 14501"

Design area validated: Abbreviated 'tid' notation. Tests that the extractor recognises the shorthand 'tid' prefix as a taxon identifier signal.

media_lookup_with_synonyms

Query: "Show media records for taxon id 187759 including synonyms"

Design area validated: includeSynonyms flag. The phrase 'including synonyms' must set include_synonyms=1 in the extraction, which the router maps to includeSynonyms=1 in MediaLookupParams.

media_lookup_with_limit

Query: "Show media results with limit 10 for tid 543034"

Design area validated: Explicit limit with tid. Extractor must extract both the limit override (10) and the tid (543034) from the same query.

media_lookup_large_limit

Query: "Get 100 media items for taxon 456123"

Design area validated: Limit at the upper boundary (100 = default max). Confirms that limit=100 is accepted by MediaLookupParams.limit (ge=1, le=1000).

Clarification Case

media_lookup_invalid_tid_format

Query: "Show media for taxon ABC123"

Design area validated: Alphanumeric tid rejection. Extractor rule: if taxon/tid value includes non-numeric characters, set clarification_needed=True and ask for a numeric tid only.

8.7 Morphology List

morphology_list_all_characters

Query: "What morphological traits does MyCoPortal track?"

Design area validated: Intent disambiguation. 'Morphological traits' must route to morphology_list, not occurrence_search or taxonomy_search. include_states defaults to 0.

morphology_list_states_cap_color

Query: "What are the possible states for cap color?"

Design area validated: includeStates flag. Phrase 'possible states' triggers include_states=1. Tests the extraction rule: 'Set include_states=1 when user asks for possible states/values/options of traits'.

morphology_list_characters_system

Query: "List all morphological characters in the system."

Design area validated: Explicit listing request without states. Should route to morphology_list with include_states=0 and default pagination.

morphology_list_with_pagination

Query: "List morphological characters with limit 50 and offset 100"

Design area validated: Explicit pagination override. Extractor must extract limit=50 and offset=100 from the query and pass them through to MorphologyListParams.

8.8 Exsiccata List

exsiccata_list_smoke

Query: "list all exsiccata"

Design area validated: Domain-specific vocabulary routing. 'exsiccata' is a specialist term; tests that the planner recognises it as the exsiccata_list intent with default limit=100, offset=0.

exsiccata_list - first 5

Query: "show me the first 5 exsiccata"

Design area validated: Limit extraction from natural language. 'first 5' must be parsed as limit=5, offset=0. Validates ordinal pagination phrasing for a specialised endpoint.

9. DeepEval Quality Evaluation

In addition to functional correctness checks, the test suite runs semantic quality scoring using DeepEval GEval metrics. These tests execute only when OPENAI_API_KEY is present and are gated by the quality marker.

9.1 Metrics

Metric	Threshold	What it scores
Reasoning Quality	0.75	Planner reasoning is specific, references query terms, and justifies tool selection
Entity Extraction Accuracy	0.75	Extracted params are complete, correctly typed, and coherent with the query

9.2 Score Distribution

After all quality test cases run, the framework automatically generates a histogram of score distributions (0–1 range) for each metric bucket. The histogram is rendered as a PNG using matplotlib and attached to the Allure report. Bars are coloured green (at or above threshold) or red (below threshold).

9.3 Threshold Override

```
# Override thresholds via environment variables
export MYCO_REASONING_THRESHOLD=0.80
export MYCO_EXTRACTION_THRESHOLD=0.70
pytest -m quality
```

10. Artifact System

All tool executions produce a JSON artifact via the shared create_json_artifact helper in execution/tools/common.py. The artifact description is built by build_artifact_description(), which extracts total/returned counts, an optional title from the payload, and a request summary derived from the search parameters used.

10.1 Artifact Metadata

Metadata key	Value
data_source	MycoPortal (always)
retrieval_date	ISO date string at execution time
search_params	dict of API parameters used (tool-specific)
total_matches	Extracted from payload count field (occurrence search)
identifier	Identifier value (single-record lookups)

10.2 Functional Artifact Assertions

The functional test bucket asserts the following for every artifact produced:

- At least one artifact is created
- artifact.description is non-empty
- 'MycoPortal' appears in the description string
- artifact.metadata['data_source'] == 'MycoPortal'
- 'retrieval_date' is present in metadata
- artifact.uris is non-empty and the first URI starts with 'http'

11. Test Fixture - FakeResponseContext

Integration tests use FakeResponseContext (tests/fixtures/mock_context.py) as a drop-in replacement for the real iChatBio ResponseContext. It captures all replies and artifacts in memory, allowing assertions without a running server or real iChatBio connection.

Component	Behaviour
FakeResponseContext	Accumulates replies[] and artifacts[] in memory
FakeProcess	Context manager returned by begin_process(); accumulates log entries
FakeArtifact	Stores mimetype, description, uris, content bytes, and metadata dict

12. Extending the Agent - Adding a New Endpoint

Follow these steps to add a new MycoPortal endpoint to the agent:

1. **Add a capability doc:** Create planning/capabilities/<tool_name>.md describing the tool, its endpoint, and its key parameters.
2. **Add the intent and tool name:** Extend the IntentType and ToolName Literals in planning/models.py.
3. **Add an extraction schema:** Create a new class inheriting ExtractionBase in extraction/models.py and add it to INTENT_SCHEMA_MAP in extractor.py.
4. **Add a param model:** Create a Pydantic model with field validators in models/params.py.
5. **Add a URL builder:** Add a build_<tool>_url() method to client/api.py.
6. **Add a router case:** Add an if plan.intent == '<intent>' branch in routing/router.py.
7. **Add a tool module:** Create execution/tools/<tool_name>.py with a run_<tool>() async function.
8. **Wire the executor:** Add an if tool_name == '<tool_name>' branch in execution/executor.py._call_tool().
9. **Add test data:** Create tests/data/<tool_name>_testdata.json and register it in tests/data/loader.py ENDPOINT_FILE_MAP.